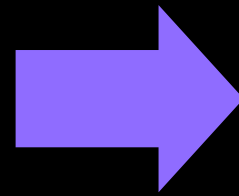
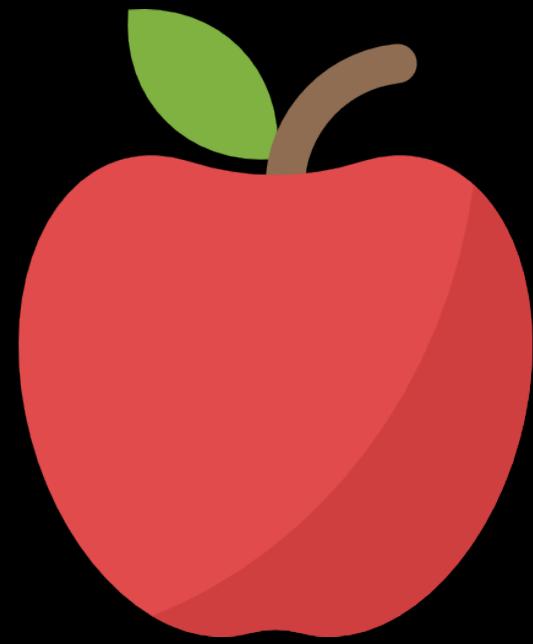


1. JavaScript 문법 복습



데이터



변수



데이터를 담는 공간



```
var fruit;           // 변수 선언  
fruit = "apple";     // 변수 초기화  
변수명      데이터  
var fruit = "apple"; // 변수 선언 및 초기화
```

변수 선언: 데이터를 담을 공간을 생성하는 것

변수 초기화: 생성된 변수에 데이터를 전달하는 것



변수 데이터 변경

```
var fruit = "apple";  
  
fruit = "banana";
```

변수 **fruit**의 데이터를 "apple"에서 "**banana**"로 변경
변수는 이미 선언되었으므로 var 키워드를 다시 작성할 필요가 없다.



변수 안의 데이터 확인 방법

```
var fruit = "apple";  
  
console.log(fruit);
```

`console.log();` 는 변수 안에 데이터를 확인할 때 사용하는 명령어



자바스크립트 사용 방법

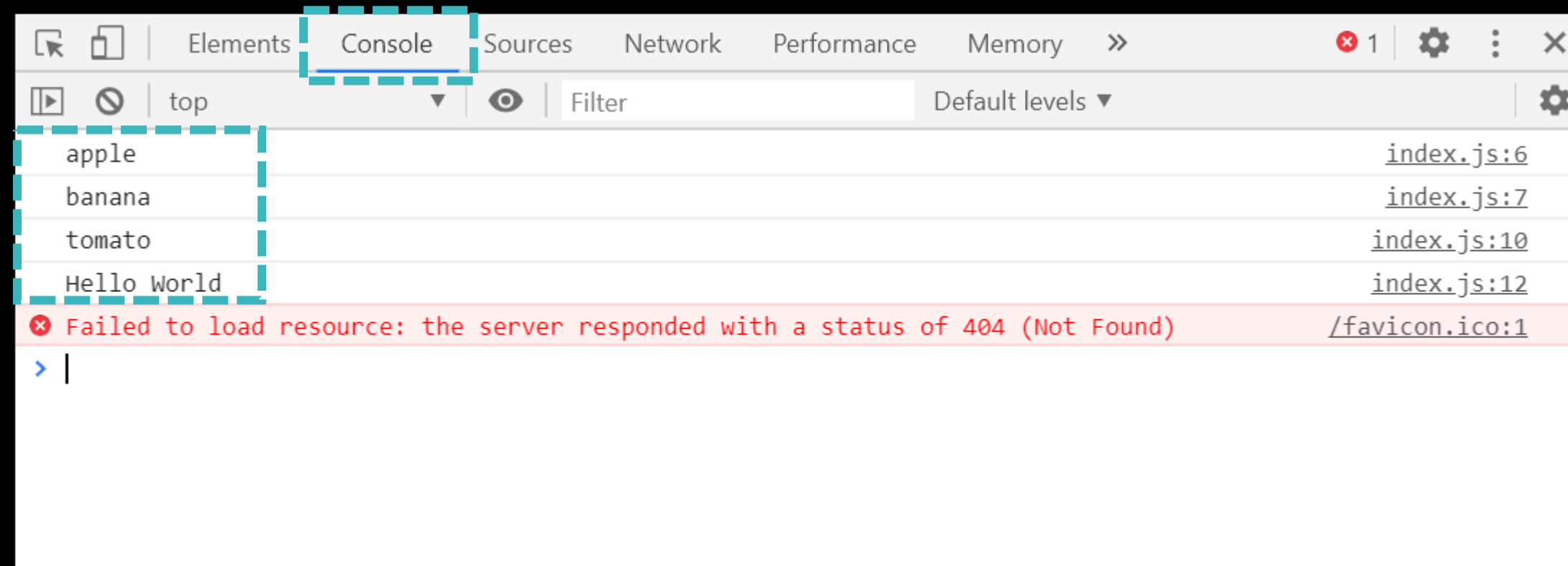
```
<body>  
  <script src="index.js"></script>  
</body>
```

script 태그의 `src`의 속성값에 JavaScript 파일 경로를 지정

→ HTML 파일과 연동



자바스크립트 사용 방법



브라우저 개발자 도구를 통해 출력된 내용 확인

(개발자 도구 열기: F12 or Windows: Ctrl + Shift + I / Mac: Cmd + Option + I)



데이터 타입

String

문자열

Number

숫자

Function

함수

Array

배열

Object

객체

Boolean

불린

undefined

정의되지 않음

null

널



```
var str1 = "Hello World";  
var str2 = "Nice to meet you";  
var str3 = "20";
```



```
var num1 = 10;    // 정수  
var num2 = -10;   // 음수  
var num3 = 3.14;  // 실수
```



함수

```
var func1 = function() {  
    console.log("func1");  
} // 함수 생성  
  
func1(); // 함수 호출
```



```
var func1 = function() {  
    console.log("func1");  
} // 함수 생성  
  
func1(); // 함수 호출
```

```
function func1() {  
    console.log("func1");  
} // 함수 생성  
  
func1(); // 함수 호출
```

함수 생성: function 키워드를 사용하여 생성

함수 호출: 함수 안에 있는 코드를 실행시키고, 그 결과를 반환



```
var area = function(width, height) {  
    return width * height; 매개변수  
} 반환 값
```

```
var result = area(10, 20);  
console.log(result); 매개변수 값 전달  
  
console.log(area(10, 20));
```

매개변수: 함수 호출 시 함수에 전달되는 데이터

return(반환 값): 함수 호출 이후 반환하는 값



```
var fruit = ["사과", "배", "수박"];

console.log(fruit);    // [ "사과", "배", "수박" ]
console.log(fruit[0]); // "사과"

fruit[0] = "포도";
console.log(fruit);    // [ "포도", "배", "수박" ]
```

비슷한 성격을 갖고 있는 데이터를 하나의 변수 안에서 관리
인덱스를 활용하여 데이터 조회 (0부터 시작)



객체

```
var student = {  
  name: "elice",  
  age: 20,  
  skills: ["JS", "HTML", "CSS"],  
  sum: function (num1, num2) { return num1 + num2 }  
}  
  
console.log(student.name);    // "elice"  
console.log(student['name']); // "elice"  
  
student.name = "Park";  
console.log(student.name);    // "Park"
```

키-값 쌍

프로퍼티

메서드

프로퍼티, 메서드로 구성
키-값 형태로 데이터 저장
여러 종류의 데이터 타입 삽입 가능



undefined, null

```
var unde;  
var empty = null;  
  
console.log(unde); // undefined  
console.log(empty); // null
```

undefined: 변수 안에 데이터를 저장하지 않은 상태

null: 개발자가 의도적으로 변수가 비어 있음을 나타낸 상태



Boolean

```
var t = true;  
var f = false;
```



내장 프로퍼티와 메서드

```
var str1 = "Hello World";

str1.length;      // 11
str1.charAt(1);   // "e"
str1.split(" ");  // [ "Hello", "World" ]
```

문자열 프로퍼티와 메서드



내장 프로퍼티와 메서드

```
var fruit = ["사과", "배", "포도"];

fruit.length;           // 3

fruit.push("딸기");      // 배열 뒤에 데이터 삽입
fruit.unshift("레몬");    // 배열 앞에 데이터 삽입

fruit.pop();             // 배열 뒤의 데이터 제거
fruit.shift();           // 배열 앞의 데이터 제거
```

배열 프로퍼티와 메서드



내장 프로퍼티와 메서드

```
Math.abs(-3);      // 절대값  
Math.ceil(0.3);    // 올림  
Math.floor(10.9);  // 내림  
Math.random();     // 임의의 숫자 출력
```

Math의 수학 연산 메서드



내장 프로퍼티와 메서드

```
parseInt("20.6");    // 정수 형태의 20 변환  
parseFloat("20.6");  // 실수 형태의 20.6 변환
```

문자를 숫자로 변환하는 메서드



산술 연산자

```
console.log(20 + 10); // 30  
console.log(20 - 10); // 10  
console.log(20 * 10); // 200  
console.log(20 / 10); // 2  
console.log(20 % 10); // 0
```



산술 연산자

```
console.log("20" + "10"); // "2010"  
console.log("20" - "10"); // 10  
console.log("20" * "10"); // 200  
console.log("20" / "10"); // 2  
console.log("20" % "10"); // 0
```

숫자 뿐 아니라 문자열도 산술 연산자 사용 가능



증감 연산자

```
var num = 10;
```

```
console.log(++num); // num + 1 후 num 출력
```

```
console.log(--num); // num - 1 후 num 출력
```

```
console.log(num++); // num 출력 후 num + 1
```

```
console.log(num--); // num 출력 후 num - 1
```




비교 연산자

```
console.log(10 == 20);    // 값이 같다
console.log(10 === 20);   // 데이터 타입과 값이 같다
console.log(10 !== 20);   // 데이터 타입과 값이 같지 않다

console.log(10 > 20);      // false
console.log(10 >= 20);     // false
console.log(10 < 20);      // true
console.log(10 <= 20);     // true
```




논리 연산자

// 앞 뒤 조건 모두 참인 경우에만 true를 반환하는 AND 연산자

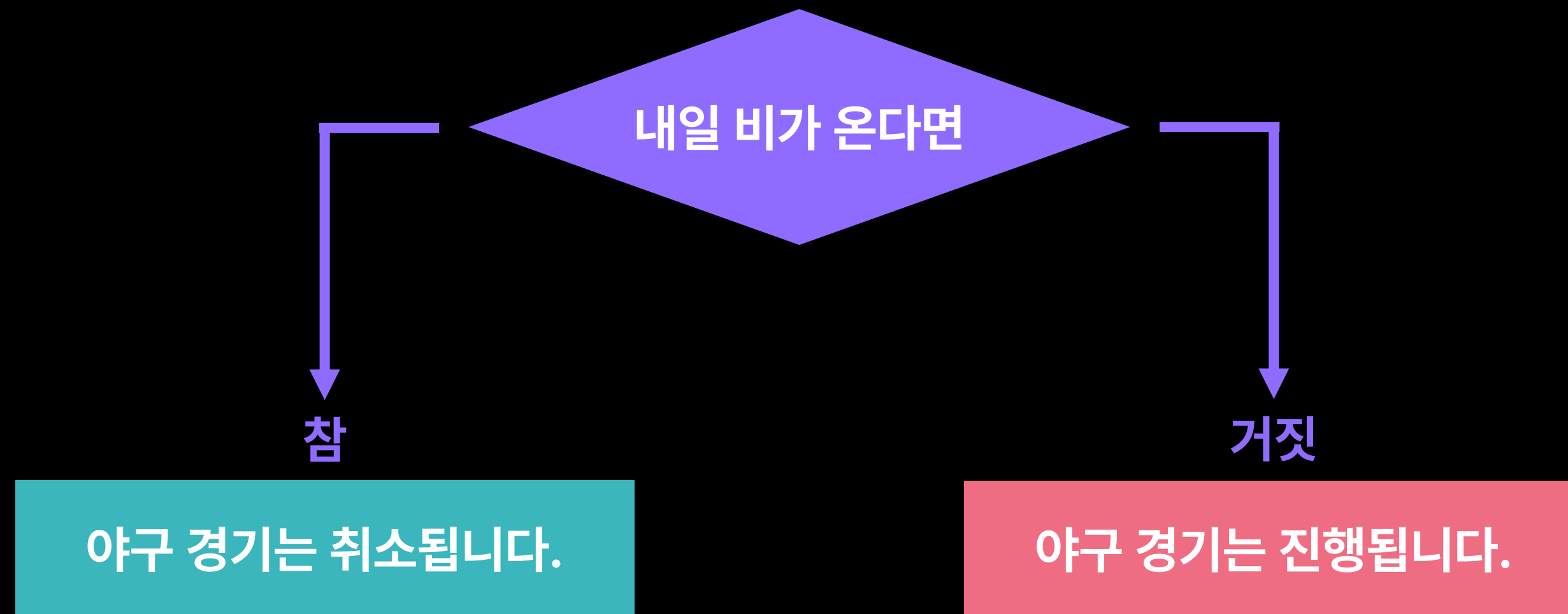
```
console.log(10 === 10 && 20 === 30); // false
```

// 둘 중 하나만 참이어도 true를 반환하는 OR 연산자

```
console.log(10 === 10 || 20 === 30); // true
```

// boolean 값을 뒤집는 NOT 연산자

```
console.log(!false); // true
```

주어진 조건에 따라 결과값을 출력하는 구문
조건으로는 비교 연산자 또는 논리 연산자 사용



```
var a = 20;  
var b = 40;  
  
if ( a < b ) {  
    console.log("a는 b보다 작다.");  
}
```



if-else문

```
var a = 20;  
var b = 40;  
  
if ( a > b ) {  
    console.log("a는 b보다 크다.");  
} else {  
    console.log("a는 b보다 작거나 같다.");  
}
```



if-else if-else문

```
var a = 20;
var b = 40;
var c = 60;

if ( a > b ) { console.log("a는 b보다 크다.");
} else if ( b > c ) { console.log("b는 c보다 크다.");
} else if ( a < c ) { console.log("a는 c보다 작다.");
} else if ( b < c ) { console.log("b는 c보다 작다.");
} else { console.log("모든 조건을 만족하지 않는다.");
}
```




```
var a = 20;
var b = 40;

if ( a !== b ) {
    if (a > b) { console.log("a는 b보다 크다"); }
    else { console.log("a는 b보다 작다"); }
} else { console.log("a와 b는 같다"); }
```




반복문 - for문

```
for (var i = 0; i < 10; i++) {  
    console.log(i);  
}
```



반복문 - while문

```
var num = 0;

while (num < 10) {
    console.log(num);
    num++;
}
```



반복문 - do-while문

```
var i = 12;  
  
do {  
    console.log(i);  
    i++;  
} while (i < 10);
```